Homework 3: Regression Review

STAT 253

Exercises

General notes for preceptors

- Students are **not allowed to use GenAI or other online resources** (besides our course website, videos, and textbook) for this assignment. If you suspect they may have done so (because of the formatting of their written response, because their written response uses terminology not covered in our class, because their code uses syntax not covered in our class, etc.) please mark the question "not yet" and get in touch with the instructor.
- Relatedly, students were asked to write all answers to these questions **in their own words**. If you suspect that any part of a student's answer may have been copied directly from the course materials or another resource like ChatGPT, please mark the question "not yet" and get in touch with the instructor.
- Students were also asked to use **only 1-2 complete sentences for each concept**. If the student's answer is longer than this, please assign (at maximum) a score of "almost" and encourage the student to think about how to explain the concept more concisely.

Exercise 0: Important Notes

Please review the **Important Notes** in the HW3 instructions document. Type your name here to acknowledge that you have read those notes and will adhere to the directions outlined therein.

Answer

For this exercise, you can provide a mark of "correct" as long as the student typed their name. If we have reason to suspect they may not have followed the HW directions/policies based on their answer to other questions, the instructor may later update their score for this exercise.

Exercise 1: Big Picture

There are some big picture concepts/ideas that I hope you remember years from now (at least in general terms), regardless of whether you end up in a data-related career. Address these concepts below. Details:

- Use 1-2 complete sentences for each concept. Brevity is a skill.
- Don't simply try to locate these ideas in your materials. Locating is different than learning.

Part a

What is the importance of "model evaluation"? NOTE: You should also think about (but not write out) the 4 important questions we should ask and how to answer each of them.

> **Answer**
>
> It's important to understand the quality of our model before reporting / using it.

Part b

What is "overfitting" and why is it bad?

> **Answer**
>
> Overfitting is when we make our model really specific to our data. That's bad because it won't extend/generalize well (in terms of predictions, summaries, etc) to new data points outside our sample.

Part c

What is "cross-validation" and what problem(s) is it trying to address? NOTE: Focus on the general concept. You should think about but NOT write out the steps of the algorithm here.

> **Answer**
>
> Cross-validation uses "training" and "testing" to evaluate how well our model predicts the outcomes of data points outside our sample. This gives us a better sense of how the model will perform on new data points (which is often what we care about!) and addresses the issue that in-sample model evaluation metrics can be overly optimistic about model performance. It can also help us prevent overfitting.
>
> NOTE: A "correct" student response should address all of these points. If they correctly address some but not all, assign a score of "almost".

Part d

What do "bias" and "variance" mean in the "bias-variance trade-off" and how do they relate to "overfitting"?

> **Answer**
>
> Bias is how well our model tends to describe the relationship of y vs x within any possible dataset. (How close are the predictions to the truth in our data.)
>
> Variance is the degree to which our model estimate, hence predictions, might vary from dataset to dataset.

Part e

When do we perform a supervised vs unsupervised learning algorithm?

> **Answer**
>
> Supervised: when we have an outcome we want to predict

Unsupervised: when we don't have an outcome in mind, we are looking for patterns in the data

Within supervised learning, when do we use a regression vs a classification algorithm?

Answer

Regression: Quantitative outcome

Classification: Categorical outcome

Exercise 2: Algorithm Pros & Cons

General notes for preceptors

Students were provided with the following instructions.

If a student's answer does not meet all criteria listed below, please provide (at maximum) a score of "almost" and provide feedback on which aspect of the instructions they have not met.

In your own words, summarize at least 1 pro and at least 1 con about some of the model building algorithms we learned in Units 1–3. Details:

- You cannot use the same pro or con more than once.
- Each pro and con must be *meaningful* and demonstrate understanding of the algorithm. For example, "this algorithm allows us to model y by x" would not suffice as a pro.
- Use only **1 complete sentence** for each pro, and 1 complete sentence for each con. Longer isn't better.
- Don't simply try to locate these ideas in your materials. Locating is different than learning.
- If it's helpful, you might want to frame your pro(s) and con(s) for a specific algorithm *relative to* another algorithm. If you do this, make sure it's clear which algorithm you are comparing to.

Part a: least squares

Answer

Pro(s):

- it has coefficients that tell us about the relationship of y vs x

Con(s):

- it doesn't help us with variable selection, thus might be overfit
- it can be too rigid to capture some relationships

Part b: LASSO

Answer

Pro(s):

- it helps us do variable selection
- (and thus, for the right choice of λ, less likely to be overfit than least squares)
- it's less variable than the least squares

Con(s):

- the coefficients lose some meaning
- the algorithm is tougher to understand
- it can be too rigid to capture some relationships

Part c: KNN

Answer

Pro(s):

- it's intuitive
- it's nonparametric / flexible

Con(s):

- it's blocky / not smooth
- curse of dimensionality (might not perform well with lots of predictors)
- lack of interpretability / information about the relationships between y and x (just gives predictions)

Part d: LOESS

Answer

Pro(s):

- it's nonparametric / flexible
- often less block / more smooth than KNN

Con(s):

- does not work if you have a categorical predictor or more than one predictor
- the information provided about the relationships between y and x is less concrete than linear regression (there are no coefficients that succinctly summarize the direction and magnitude of the relationship)

Part e: Splines

Answer

Pro(s):

- it's parametric so we can use it within linear regression (and maintain some interpretability)
- it provides flexiblility

- not too computationally intensive
- can model local trends (piece-wise)
- smooth

Con(s):

- hard to pick degrees of freedom for each variable separately (hard to tune)
- can have high variance in the boundaries/extreme (although *natural splines* address this)
- the algorithm is tougher to understand

Exercise 3: ML Lingo

General notes for preceptors

Students were provided with the following instructions.

If a student's answer does not meet all criteria listed below, please provide (at maximum) a score of "almost" and provide feedback on which aspect of the instructions they have not met.

In your own words, describe what each of these "everyday" words means *in the context of ML algorithms*. Details:

- Use only 1–2 complete sentences for each concept. Longer isn't better.
- Don't simply try to locate these ideas in your materials. Locating is different than learning.

Part a: greedy

Answer

An algorithm is "greedy" when it makes the best *local* decisions which might not end up being *globally* optimal. (example = backward stepwise selection)

Part b: parsimonious

Answer

A model is parsimonious if it contains as few predictors as possible (or is as simple as possible) while still being a good model (eg: producing accurate predictions).

Part c: goldilocks problem

Answer

When tuning an algorithm, we don't want our tuning parameter to be too small or too big, but just right. When the tuning parameter is too small / too large, our model might produce inaccurate predictions.

Exercise 4: R Code

We've used a lot of `tidymodels` code to build and apply our ML algorithms. As much as it is, the general structure of this code is similar from algorithm to algorithm. Let's connect the dots. Challenge yourself to do as much as possible without your notes – locating is different than learning.

Part a

The code structure below represents the first 3 steps in building LASSO and KNN models. (The code is *similar* for other algorithms we've discussed.) Directions:

- Fill in each `___` (and think about what it does). Where appropriate, use the objects defined earlier in the chunk (eg: `my_spec` not `lm_spec`).
- The `aaa`, `bbb`, and `ccc` represent components that will be different for LASSO and KNN. Do not remove these.
- Do not remove `eval = FALSE`. This is example code that will not run!

Answer

```
# Specify the algorithm
my_spec <- aaa() %>%
  set_mode("regression") %>%
  set_engine(bbb) %>%
  set_args(ccc)

# Construct the variable recipe
# Include steps to (1) convert categorical predictors to dummy variables ; and (2) standardize qu
my_variable_recipe <- recipe(y ~ ., data = my_data) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())

# Set up the modeling workflow
# NOTE: the order of the model and recipe doesn't matter
my_workflow <- workflow() %>%
  add_recipe(my_variable_recipe) %>%
  add_model(my_spec)
```

Part b

Specify the `aaa`, `bbb`, and `ccc` components for the LASSO and KNN.

aaa: algorithm type

Answer

- LASSO: `linear_reg()`

- KNN: `nearest_neighbor()`

bbb: engine / function we'll use to build the algorithm

Answer

- LASSO: `"glmnet"`
- KNN: `"kknn"`

ccc: algorithm-specific arguments needed to run the function

Answer

- LASSO: `mixture = 1, penalty = tune()`
  - NOTE: the `mixture = 1` part specifies that we want LASSO, not ridge or elastic net; the `penalty = tune()` part is where we're saying that we want to tune the λ parameter
- KNN: `neighbors = tune()`

Part c

The below code chunk is written in a generic form for many of our models. Explain what this code would produce / what the point of this code is. Keep your answer to 1-2 sentences, be specific (eg: make clear what the 75 and 8 mean in the code below), and do not modify the code in any way.

```
set.seed(___)
my_models <- my_workflow %>%
  tune_grid(
    grid = grid_regular(___(range = c(___, ___)), levels = 75),
    resamples = vfold_cv(my_data, v = 8),
    metrics = metric_set(mae)
  )
```

Answer

Responses should be between 1 and 2 sentences and address both of the following points:

- This runs **75** models, each using a different value for the tuning parameter.
- Each of these are evaluated using **8-fold** CV.

Other things that could be noted here:

- This code calculates MAE on each test fold. If you want a different evaluation metric, you'd need to update the `metric_set`.
- The first blank in the `grid_regular` function should be replaced by the name of the tuning parameter (eg `penalty` for LASS).
- The next two blanks are the lower and upper bound of values you want to try for that parameter. In the case of LASSO, this will be specified on a log scale (eg `c(-5, 1)` means you want to try λ between 10^{-5}

and 10^1). For KNN, the range is as specified (eg `c(1, 10)` means you want try K between 1 and 10).
- We need the `set.seed` function because cross validation is a random algorithm — points are randomly assigned to folds. Without setting a seed, you would get different results each time you run this code.

Part d

The code structure below represents the finalization of the algorithm. Directions:

- Fill in each `___` (and think about what it does).
- Where relevant, use the object names we defined in Parts a and c (eg: `my_workflow`)
- Do not remove `eval = FALSE`. This is example code that will not run!

Answer

```
# Identify the tuning parameter which produced the lowest CV MAE
my_parameter <- my_models %>%
  select_best(metric = "mae")

# Finalize the algorithm using this tuning parameter
my_final_model <- my_workflow %>%
  finalize_workflow(parameters = my_parameter) %>%
  fit(data = my_data)
```

Exercise 5: Connections

For each pair of algorithms below, identify: (1) a key *similarity*; (2) a key *difference*; and (3) only if relevant, any scenario in which they're "equivalent". Challenge yourself to go as far beyond the surface as possible.

Part a: KNN vs LOESS

Answer

Similarity:

- Both nonparametric
- both allow for flexibility
- both look at local areas
- both use weighted averaging (points closer to the prediction get more highly weighted)

Difference:

- local area defined by number of neighbors (KNN) vs span (LOESS)
- average information in local area (KNN) vs. use linear or quadratic regression models in local area (LOESS)

Never equivalent

Part b: LOESS vs splines

Answer

Similarity:

- Both allow for flexibility
- polynomials in local areas

Difference:

- Nonparametric (LOESS) v. parametric (splines)
- moving window (LOESS) v. fixed window (splines)

Never equivalent

Part c: splines vs least squares

Answer

Similarity:

- Can use splines with least squares

Difference:

- You don't have to use splines in least squares

Equivalence doesn't make sense here

Part d: least squares vs LASSO

Answer

Similarity:

- Linear regression assumptions
- parametric

Difference:

- LASSO does variable selection

Equivalent if lambda is 0

Exercise 6: Resource Reflection

List the resources you used to complete this assignment (e.g. office hours, textbook, Gen AI, etc.)

-
-
-

Write 3-5 sentences about which resources were the most useful in helping you complete the assignment.

Answer

Answers will vary, but students should list the resouces they used to complete this assignment (e.g. office hours, textbook, Gen AI, etc.) and write 3-5 sentences about which resources were the most useful in helping them complete the assignment.

IMPORTANT NOTE: GenAI usage is *NOT* allowed for this particular assignment. If a student mentions a GenAI resource on their assignment, please let the instructor know.